

Tunisian Republic
Ministry of Higher Education and Scientific Research

Tunis Business School
Department of Information Technology



End-of-Semester Project

Course: **IT360: Information Assurance and Security**

Field of Study: **Information Technology**

Theme

Text Encryption Using Cryptographic Algorithms

Cybersecurity Project — Part 1

Sections covered: Core Cryptographic Concepts; Existing Solutions

Presented by

Mohamed Mouldi Bziou

Mohamed Yassine Jemli

Tasnim Ben Dhia

Chaima Khiari

Achref Khaled

Submitted in: **April 2026**

Academic Year: 2025/2026

0 Contents

1	Main Concepts of Text Encryption	2
1.1	Introduction to Cryptography	2
1.2	Fundamental Terminology	2
1.3	The General Encryption Flow	3
1.4	Categories of Cryptographic Algorithms	3
1.4.1	Symmetric Encryption	3
1.4.2	Asymmetric Encryption	5
1.4.3	Hash Functions	5
1.5	Symmetric vs. Asymmetric: Comparative Overview	6
1.6	Deep Dive: Fernet (AES-128-CBC + HMAC-SHA256)	7
1.7	Deep Dive: RSA Encryption	8
1.8	Key Management Principles	8
2	State of the Art: Existing Solutions	9
2.1	Overview of Existing Encryption Libraries and Tools	9
2.2	Low-Level Cryptographic Libraries	9
2.2.1	OpenSSL	9
2.2.2	libsodium	9
2.2.3	PyCryptodome	10
2.2.4	Bouncy Castle	10
2.3	High-Level Libraries and Frameworks	10
2.3.1	Python <code>cryptography</code> Library (Fernet)	10
2.3.2	Web Crypto API	10
2.3.3	Java Cryptography Architecture (JCA)	10
2.4	Cloud Key Management and Encryption-as-a-Service	10
2.5	End-User Encryption Tools	11
2.6	Comparative Analysis of Main Algorithms	11
2.7	Equivalent Security Key Sizes (NIST SP 800-57)	11
2.8	Summary and Motivation for the Proposed Solution	12

1 Main Concepts of Text Encryption

1.1 Introduction to Cryptography

Definition — Cryptography

Cryptography is the science of securing information by transforming it into an unreadable format, ensuring **confidentiality**, **integrity**, **authenticity**, and **non-repudiation** of data in transit or at rest.

Cryptography has evolved over thousands of years — from simple Caesar ciphers in antiquity to mathematically rigorous algorithms that underpin modern digital communication. The core goal remains unchanged: allow legitimate parties to communicate secretly while preventing eavesdroppers from deciphering the message.

The three classical security goals addressed by encryption are:

- **Confidentiality** — only authorized parties can read the plaintext.
- **Integrity** — the message cannot be altered without detection.
- **Authenticity** — the origin of the message can be verified.

1.2 Fundamental Terminology

Table 1: Core cryptographic terminology

Term	Definition
Plaintext	The original, human-readable message before encryption.
Ciphertext	The scrambled, unreadable output produced by the encryption algorithm.
Key	A secret value (string of bits) that controls the encryption/decryption process.
Algorithm	The mathematical function that transforms plaintext to ciphertext and vice versa.
Encryption	The process of converting plaintext into ciphertext using a key.
Decryption	The reverse process — recovering plaintext from ciphertext using a key.
Key Space	The set of all possible keys for a given algorithm.
IV / Nonce	Random value preventing identical outputs for identical inputs.
Salt	Random data added to input before hashing to prevent rainbow-table attacks.

1.3 The General Encryption Flow

Figure 1 illustrates the general encryption and decryption pipeline.

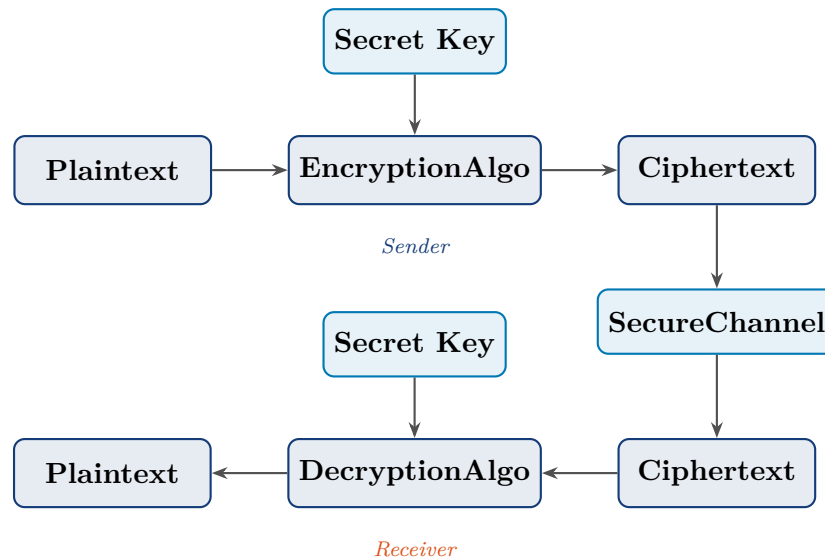


Figure 1: General encryption and decryption flow

1.4 Categories of Cryptographic Algorithms

Modern cryptography is generally divided into three families, as shown in Figure 2.

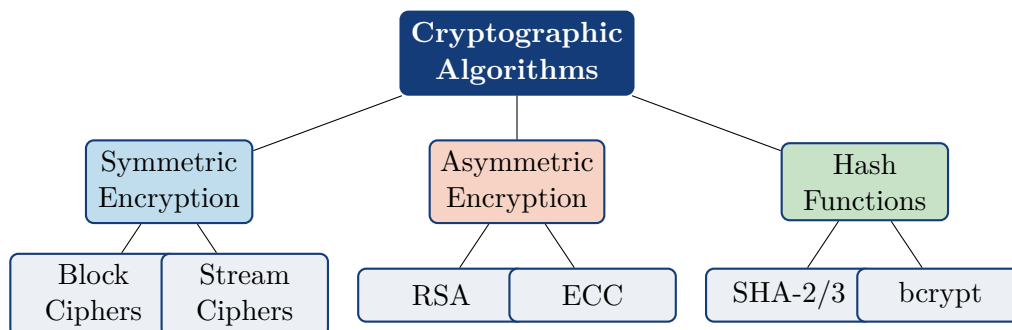


Figure 2: Taxonomy of modern cryptographic algorithms

1.4.1 Symmetric Encryption

Symmetric Encryption

Both the sender and receiver share a **single secret key** for encryption and decryption. The key must be exchanged via a secure channel before communication begins.

Block Ciphers. A block cipher splits the plaintext into fixed-size blocks (e.g., 128 bits) and encrypts each block. The two most important block ciphers are:

- **AES (Advanced Encryption Standard)** — operates on 128-bit blocks with keys of 128, 192, or 256 bits. It replaced DES and is the global standard for symmetric encryption.

- **DES / 3DES** — older 64-bit block cipher, now deprecated for most uses.

Block ciphers require a *mode of operation* to handle messages longer than one block (Table 2).

Table 2: Common block cipher modes of operation

Mode	IV needed	Parallelisable	Notes
ECB	No	Yes	Insecure — identical blocks $\Rightarrow$ identical ciphertext
CBC	Yes	Decrypt only	Used in Fernet; each block XOR'd with previous ciphertext
CTR	Yes	Yes	Turns block cipher into a stream cipher
GCM	Yes	Yes	Authenticated encryption: integrity + confidentiality
CFB	Yes	Decrypt only	Self-synchronising stream mode

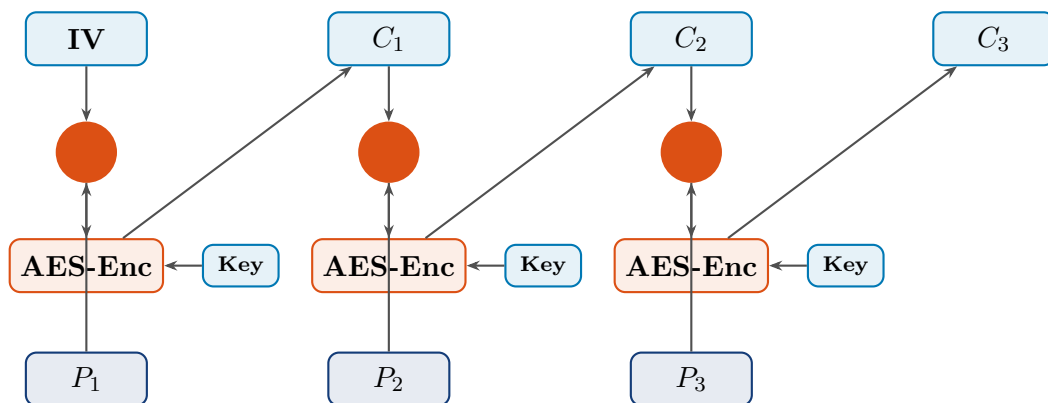


Figure 3: AES in CBC mode: each block depends on the previous ciphertext block

AES-CBC mode — chaining illustration.

Stream Ciphers. Stream ciphers encrypt data byte-by-byte using a keystream derived from the secret key. Notable examples: **ChaCha20** and RC4 (deprecated).

1.4.2 Asymmetric Encryption

Asymmetric (Public-Key) Encryption

Uses a mathematically linked **key pair**: a *public key* (freely shared) for encryption and a *private key* (kept secret) for decryption. No prior secure key exchange is required.

RSA (Rivest–Shamir–Adleman). RSA is based on the computational hardness of factoring the product of two large primes. Key generation steps:

1. Choose two large primes p and q . Compute $n = p \cdot q$.
2. Compute Euler's totient: $\phi(n) = (p - 1)(q - 1)$.
3. Choose e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
4. Compute $d \equiv e^{-1} \pmod{\phi(n)}$.
5. **Public key:** (n, e) ; **Private key:** (n, d) .

$$C = M^e \bmod n \quad (\text{Encryption})$$

$$M = C^d \bmod n \quad (\text{Decryption})$$

ECC (Elliptic Curve Cryptography). ECC provides equivalent security to RSA with much shorter keys (256-bit ECC $\approx$ 3072-bit RSA), making it preferable for resource-constrained devices.

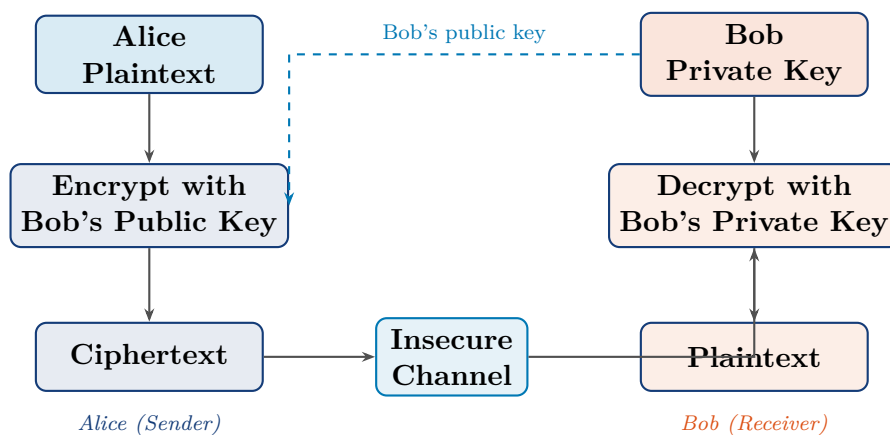


Figure 4: Asymmetric encryption flow (RSA / ECC)

1.4.3 Hash Functions

Hash functions produce a fixed-size *digest* from arbitrary-length input. They are **one-way** — recovering the input from the hash is computationally infeasible. Key properties:

- **Deterministic** — same input always gives the same hash.
- **Pre-image resistance** — given h , infeasible to find m with $H(m) = h$.

- **Collision resistance** — infeasible to find $m_1 \neq m_2$ with $H(m_1) = H(m_2)$.
- **Avalanche effect** — a tiny input change drastically alters the output.

Common algorithms: **SHA-256**, **SHA-3**, **bcrypt** (password hashing with a configurable work factor).

1.5 Symmetric vs. Asymmetric: Comparative Overview

Table 3: Symmetric vs. Asymmetric encryption — key differences

Criterion	Symmetric	Asymmetric
Key count	1 shared key	2 keys (public + private)
Key exchange	Requires secure channel	No prior exchange needed
Speed	Very fast (hardware-accel.)	Slow ($10\times$ – $1000\times$ slower)
Key size	128–256 bits	2048–4096 bits (RSA)
Use case	Bulk data encryption	Key exchange, digital signatures
Example	AES, ChaCha20, Fernet	RSA, ECC, ElGamal
Security basis	Key secrecy	Mathematical hard problem

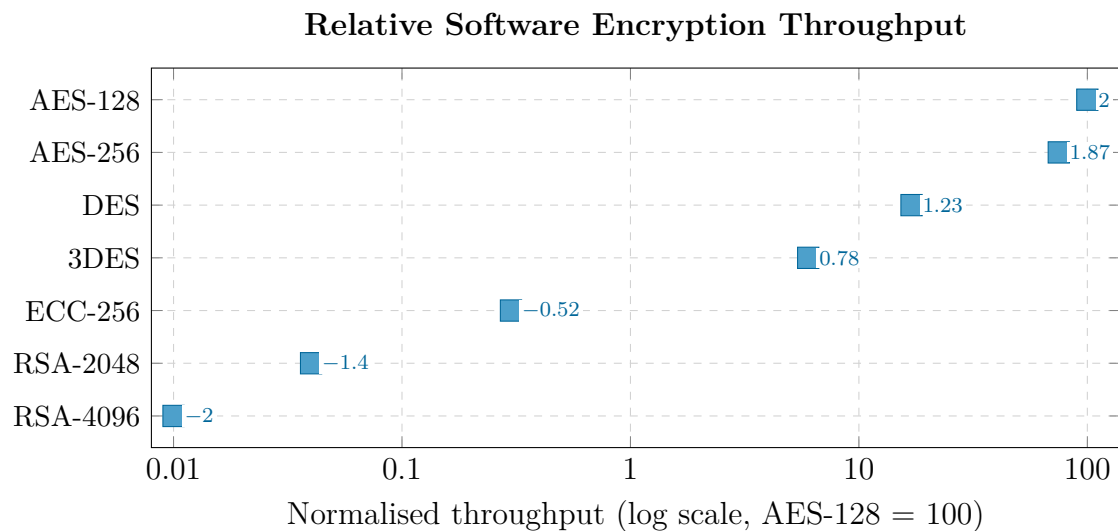


Figure 5: Indicative normalised throughput (AES-128 = 100). RSA/ECC values are dramatically lower than symmetric ciphers.

1.6 Deep Dive: Fernet (AES-128-CBC + HMAC-SHA256)

Fernet — Definition

Fernet is a high-level symmetric authenticated encryption scheme from the `cryptography` Python library. It guarantees that a message encrypted using it cannot be manipulated or read without the key, and it always produces a different ciphertext even for the same plaintext.

Internal structure of a Fernet token. A Fernet key is 32 bytes: 16 bytes for AES-128-CBC encryption and 16 bytes for HMAC-SHA256 signing. The token layout is:

$$\underbrace{\text{Version}}_{1\text{ B}} \parallel \underbrace{\text{Timestamp}}_{8\text{ B}} \parallel \underbrace{\text{IV}}_{16\text{ B}} \parallel \underbrace{\text{Ciphertext}}_{\text{variable}} \parallel \underbrace{\text{HMAC}}_{32\text{ B}}$$

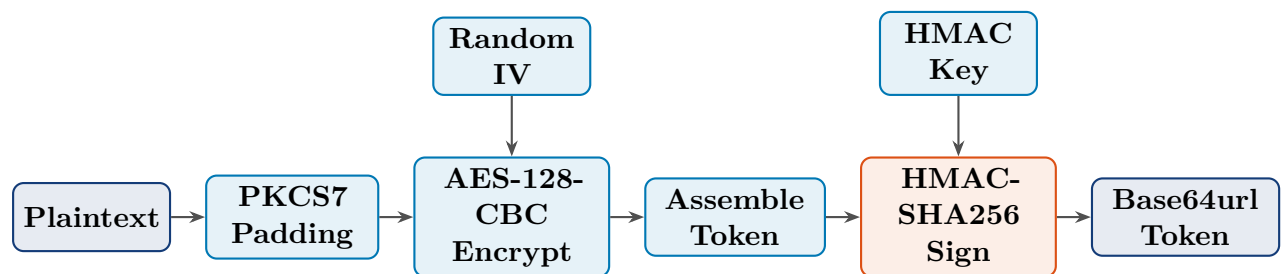


Figure 6: Fernet encryption pipeline

Functional flow.

1.7 Deep Dive: RSA Encryption

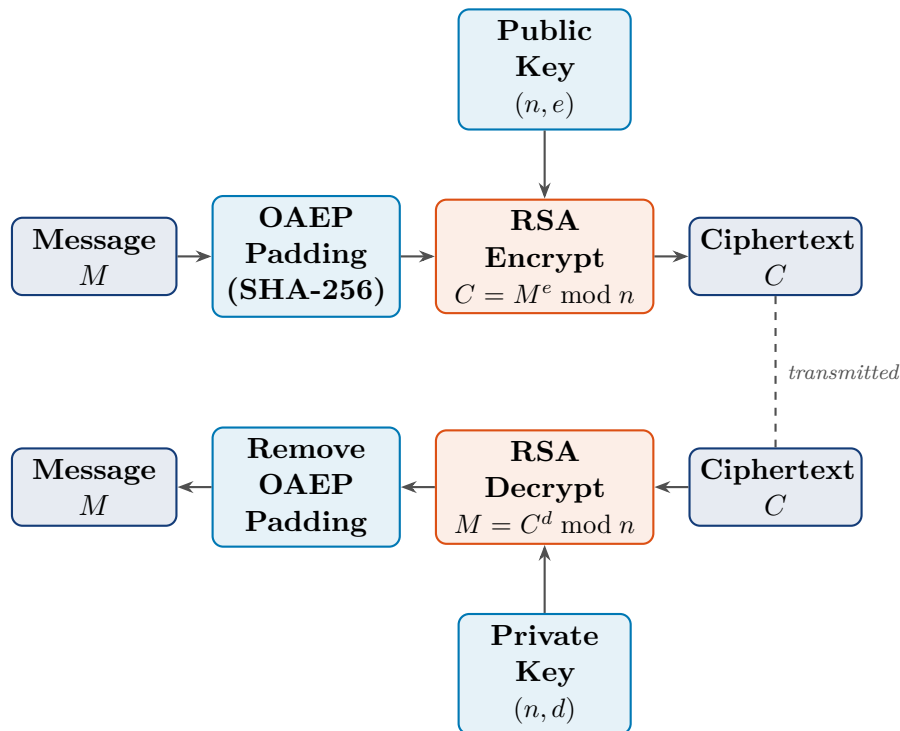


Figure 7: RSA encryption and decryption with OAEP padding

1.8 Key Management Principles

Strong algorithms are useless without proper key management. The main concerns are:

- **Key Generation** — use a cryptographically secure random number generator (CSPRNG).
- **Key Storage** — never store keys in plaintext; use HSMs or key-derivation functions (PBKDF2, Argon2).
- **Key Exchange** — use Diffie-Hellman (DH) or Elliptic-curve DH (ECDH).
- **Key Rotation** — regularly replace keys to limit exposure.
- **Key Revocation** — revoke and destroy compromised keys immediately.

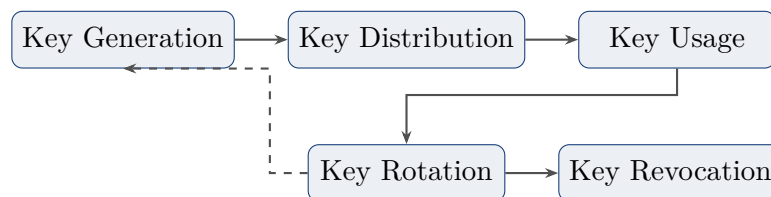


Figure 8: Key lifecycle

2 State of the Art: Existing Solutions

2.1 Overview of Existing Encryption Libraries and Tools

The landscape of text encryption tools spans from low-level cryptographic libraries to high-level web APIs. Figure 9 maps the major solutions by abstraction level and primary use case.

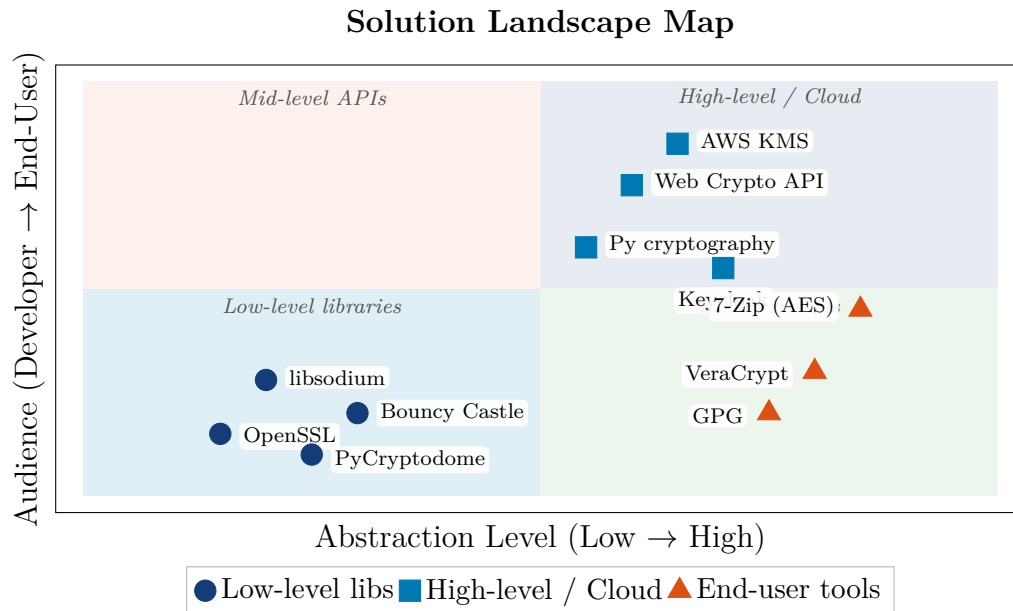


Figure 9: Mapping of existing encryption solutions by abstraction level and target audience

2.2 Low-Level Cryptographic Libraries

2.2.1 OpenSSL

OpenSSL is the most widely deployed open-source cryptographic library, providing implementations of SSL/TLS, AES, RSA, ECC, and hash functions in C with bindings for most languages.

Key Facts — OpenSSL

- License: Apache 2.0 (OpenSSL 3.x)
- Supports: AES, RSA, ECDSA, SHA-2/3, ChaCha20, X.509
- FIPS 140-2 validated module available
- Used by: Apache, nginx, OpenSSH, curl

2.2.2 libsodium

libsodium is a modern, portable library offering authenticated encryption (XSalsa20-Poly1305), public-key cryptography (Curve25519 / Ed25519), and key exchange. Its design philosophy favours safe defaults over configurability.

2.2.3 PyCryptodome

A self-contained Python package providing AES, RSA, DSA, SHA, and MD5. It is the de-facto standard for pure-Python cryptographic projects.

2.2.4 Bouncy Castle

A comprehensive Java/C# cryptographic API, underlying engine for many enterprise security frameworks.

2.3 High-Level Libraries and Frameworks

2.3.1 Python cryptography Library (Fernet)

The `cryptography` package offers both a low-level *hazmat* layer and the high-level *Fernet* recipe that handles AES-128-CBC, HMAC, IV generation, and Base64url encoding automatically. Recommended for most Python projects.

2.3.2 Web Crypto API

A W3C standard built into all modern browsers, providing:

- AES-GCM (recommended), AES-CBC, AES-CTR
- RSA-OAEP, RSA-PSS, ECDH, ECDSA
- Key derivation: PBKDF2, HKDF

All operations are asynchronous and hardware-accelerated where available.

2.3.3 Java Cryptography Architecture (JCA)

A provider-based architecture in the Java standard library using high-level `Cipher`, `KeyPairGenerator`, and `MessageDigest` classes, with pluggable providers (Sun, BouncyCastle, etc.).

2.4 Cloud Key Management and Encryption-as-a-Service

Table 4: Cloud encryption / KMS solutions comparison

Solution	Provider	Highlights	Free Tier
AWS KMS	Amazon	HSM-backed, CMKs, envelope encryption, CloudTrail audit	Limited
Azure Key Vault	Microsoft	Secrets + keys + certs, RBAC, FIPS 140-2 Level 2	Yes
Google Cloud KMS	Google	Software + HSM keys, external key manager option	Yes
HashiCorp Vault	Open-source	Self-hosted, dynamic secrets, transit encryption	Community

2.5 End-User Encryption Tools

- **GnuPG (GPG)** — implements the OpenPGP standard for email/file encryption using RSA or ECC. Widely used for software release signing.
- **VeraCrypt** — on-the-fly disk encryption with AES, Serpent, and Twofish (optional cascading). Open-source successor to TrueCrypt.
- **Signal Protocol** — end-to-end encrypted messaging using the Double Ratchet Algorithm (X3DH + forward secrecy). Used by Signal, WhatsApp, and Messenger.
- **Age** — modern file-encryption tool (X25519, ChaCha20-Poly1305), designed as a simple GPG replacement.

2.6 Comparative Analysis of Main Algorithms

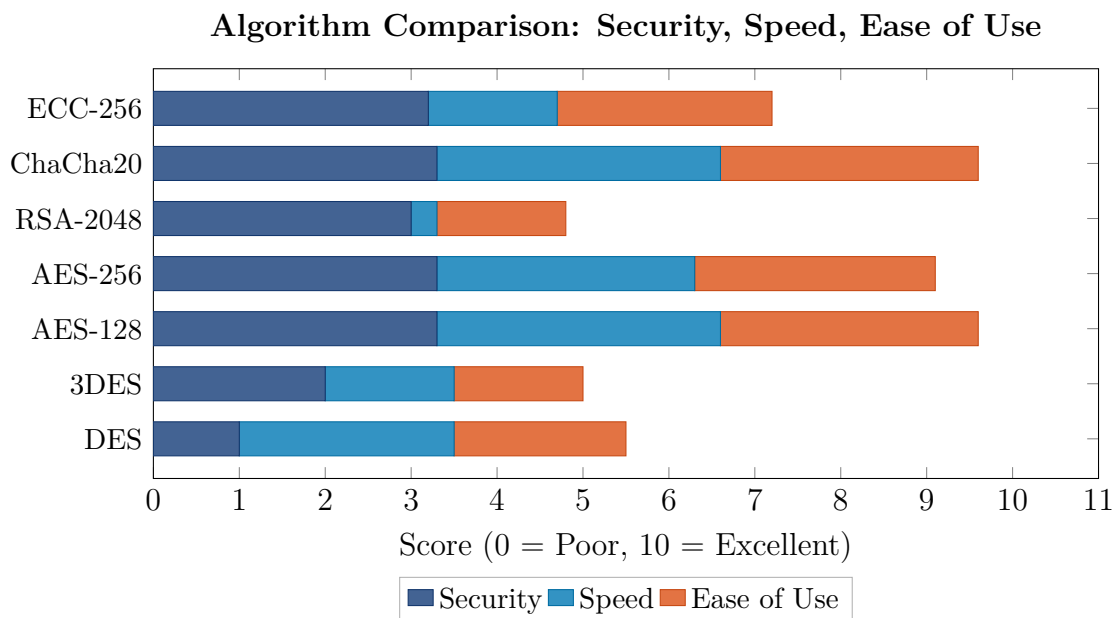


Figure 10: Qualitative comparison of common algorithms (stacked score, max 10)

2.7 Equivalent Security Key Sizes (NIST SP 800-57)

Table 5 shows the NIST-recommended key sizes needed to achieve equivalent security levels across different algorithm families.

Table 5: Key sizes for equivalent security levels (NIST SP 800-57)

Security (bits)	Symmetric (AES)	RSA / DSA	ECC (ECDSA)
80	80 bits	1024 bits	160 bits
112	112 bits	2048 bits	224 bits
128	128 bits	3072 bits	256 bits
192	192 bits	7680 bits	384 bits
256	256 bits	15360 bits	512 bits

2.8 Summary and Motivation for the Proposed Solution

Key Takeaways — State of the Art

1. **AES-128/256** in GCM or CBC mode is the gold standard for symmetric text encryption due to its speed, security, and hardware support.
2. **RSA** remains the most widely deployed asymmetric algorithm; **ECC** is the modern, compact alternative.
3. **Fernet** (Python) is the simplest correct symmetric recipe: AES-CBC + HMAC + base64 in a single safe API.
4. **Hybrid encryption** (RSA wraps a symmetric key; AES encrypts the data) is the industry standard for large-scale systems.
5. Strong encryption **must** produce different ciphertext for identical plaintext — guaranteed by a random IV/nonce.
6. Modern tools like **Signal Protocol** and **Age** prove that security and developer ergonomics can coexist.

Based on this state of the art, our proposed web application will implement:

- Symmetric encryption using **Fernet (AES-128-CBC + HMAC-SHA256)**.
- Asymmetric encryption using **RSA-2048 with OAEP padding**.
- A hybrid mode combining both for large inputs.
- A visually interactive interface that demonstrates how ciphertext changes with every encryption run due to the random IV.